

MEDISTORE AI

Intelligent Pharmacy Management & Multi-Agent AI Integration

Final Year Project - Topic Proposal Document

AI & Machine Learning Extension of an Existing Full-Stack Pharmacy Platform

Prepared by: [Your Full Name]

Student ID: [Your Student ID]

Programme: BSc (Hons) in Information Technology / Computer
Science

Base System: MediStore - Pharmacy Inventory & Operations
Platform (OOAD Group Project, Y2S1, SLIIT)

Introduction

This document presents the proposed Final Year Project topic: the design and implementation of **MediStore AI**, an intelligent extension of MediStore - a full-stack pharmacy inventory and operations platform originally developed as a group project for the Object Oriented Analysis & Design (OOAD) module. The proposed project introduces a suite of Machine Learning components and Large Language Model (LLM)-based AI agents into the existing system, transforming MediStore from a conventional CRUD-based management platform into an intelligent pharmacy assistant capable of natural-language interaction, automated reporting, document understanding, and computer-vision-based staff attendance.

Four core AI-driven modules are proposed, each addressing a distinct operational pain point in small-to-medium pharmacy management in the Sri Lankan context: (1) a Medicine Information Q&A Agent grounded in a curated drug knowledge base, (2) an AI-Augmented Reporting Agent that generates narrative insights alongside existing PDF/CSV reports, (3) a Report Analysis (RAG) Agent that allows staff to upload documents and query them in natural language, and (4) a Pharmacy Business Intelligence (BI) Agent that uses tool-calling to reason over live inventory and sales data. A complementary computer vision component - a Face Recognition-based Staff Attendance System - is also proposed, building on a prototype already developed using a webcam and a Teachable Machine-trained classifier.

Each module below is presented with a justified problem statement, research/engineering objectives, candidate datasets and tools, proposed model/architecture choices, an evaluation design, and pre-identified implementation challenges, in line with the structure expected for FYP topic proposals. A summary scoring table is provided first to give the supervisor an overview of relative scope and complexity across the proposed modules.

Summary Comparison of Proposed AI Modules

Module	Problem Strength	Data / Resource Avail.	Technical Complexity	Pharmacy Operational Impact	Avg. Score
Module 1: Medicine Information Q&A Agent (RAG-based)	8/10	8/10	5/10	9/10	7.5/10
Module 2: AI-Augmented Reporting Agent	7/10	9/10	5/10	8/10	7.3/10
Module 3: Report Analysis Agent (Document RAG)	8/10	8/10	6/10	8/10	7.5/10
Module 4: Pharmacy BI & Assistant Agent (Tool-Calling)	8/10	9/10	6/10	9/10	8.0/10
Module 5: Face Recognition Attendance System	7/10	7/10	6/10	7/10	6.8/10

Note: The Pharmacy BI & Assistant Agent (Module 4) is identified as the highest-impact component, as it directly automates decision support for reordering, expiry management, and sales analysis using data already present in the MediStore database - requiring no new data collection.

Proposed System Architecture Overview

The existing MediStore platform (React frontend, Java Spring Boot backend, MySQL database) will be re-architected as a unified Python / FastAPI backend to simplify integration of the proposed AI components. All AI agents are implemented as internal modules within the same FastAPI application, calling the existing service layer (inventory, sales, prescriptions) directly as Python functions rather than through inter-service HTTP calls. This consolidation reduces architectural complexity and allows AI agents to use function-calling/tool-use patterns to access live operational data with minimal latency.

Frontend	React (existing) - Dashboard, Chat UI for AI agents, Attendance UI, Report Analysis UI
Backend Framework	Python 3.12 + FastAPI (unified, replacing Spring Boot)
Relational Database	PostgreSQL / MySQL (SQLAlchemy async ORM, Alembic migrations)
Vector Database	Qdrant - stores embeddings for medicine knowledge base and uploaded reports
LLM Provider	Claude API / Gemini API / Groq (Llama models) via a unified llm_client wrapper
Task Scheduling	Celery + Redis - scheduled reports, low-stock and expiry alerts
Computer Vision	OpenCV (face detection) + TensorFlow Lite (Teachable Machine-exported model)
Containerisation	Docker Compose - API, worker, beat, Postgres, Redis, Qdrant, frontend

The five proposed modules are detailed in the following sections, each following the same structure used for FYP topic evaluation: problem/research gap, objectives, datasets and resources, models and techniques, selection factor scoring, pre-identified challenges, key references, and an evaluation design comprising standard metrics (Layer 1) and a contribution-specific evaluation (Layer 2).

Module 1 - Medicine Information Q&A Agent

Domain: Healthcare NLP / Retrieval-Augmented Generation (RAG) | Overall Score: 7.5/10 | Highly Recommended

Problem Statement

Pharmacy staff frequently need quick access to accurate medicine information - dosage guidelines, common usage, contraindications, and interactions - particularly for less-familiar drugs. General-purpose LLMs can answer such questions from pretrained knowledge, but this carries risk: LLMs can produce outdated, generic, or subtly incorrect dosage information, and provide no traceable source for verification. There is a clear need for a domain-grounded Q&A system that answers from a curated, verifiable medicine knowledge base specific to the medicines stocked in MediStore's inventory, with retrieval-based grounding rather than reliance on raw model recall.

At a Glance

Task type	Domain-grounded question answering (RAG) over a curated medicine knowledge base
Primary data source	Curated medicine reference dataset (dosage, indications, contraindications, interactions) compiled for medicines present in MediStore inventory
Supporting sources	OpenFDA Drug Label API, RxNorm (NLM), DrugBank (open/free tier), WHO Model List of Essential Medicines
Embedding model	Sentence-transformers (e.g. all-MiniLM-L6-v2) or LLM-provider embedding API
Vector store	Qdrant - dedicated 'medicine_kb' collection
LLM	Claude API / Gemini API (retrieval-augmented prompt with cited source chunks)
Key technique	Retrieval-Augmented Generation with mandatory source attribution and a built-in 'consult a licensed pharmacist' safety disclaimer on dosage queries
Constraint	Medical-information accuracy, source traceability, and responsible-use safeguards

Objectives

- Compile a structured medicine knowledge base (CSV/JSON) covering the medicine categories present in MediStore inventory, including generic name, dosage forms, standard adult/paediatric dosage, common indications, contraindications, and key interactions.
- Cross-reference and validate entries against OpenFDA / RxNorm / WHO EML to ensure accuracy and currency.
- Generate embeddings for each knowledge base entry and index them into a dedicated Qdrant collection.
- Implement the Medicine Q&A agent as a FastAPI endpoint that performs similarity search, constructs a grounded prompt with retrieved context, and queries the LLM API.
- Append source attribution (which knowledge-base entries informed the answer) and a standard medical disclaimer to every response.

- Evaluate answer accuracy against a held-out set of question-answer pairs derived from the knowledge base, and assess groundedness (whether answers are supported by retrieved context).

Selection Factor Scores

Factor	Score	Rating
Problem/gap relevance	8/10	Strong - addresses real operational need for pharmacy staff
Data availability	8/10	Public drug databases available; curation effort required but feasible
Technical complexity	5/10	Standard RAG pipeline - well-documented pattern
Pharmacy operational impact	9/10	Directly supports staff in daily medicine queries
Supervisor appeal	7/10	Demonstrates applied NLP/RAG competency
Novelty / contribution	6/10	Novelty lies in domain-specific curation and integration, not the RAG technique itself
Buildability in project time	9/10	Achievable within standard FYP timeline using existing LLM APIs
Literature depth	7/10	Substantial RAG and clinical-NLP literature available

Pre-Identified Challenges

- Knowledge base curation effort - compiling and validating a sufficiently broad medicine dataset (target: 100-200 entries) is time-intensive and requires careful sourcing.
- Avoiding hallucination - the agent must be constrained to answer only from retrieved context; out-of-scope queries should be explicitly declined rather than answered from general LLM knowledge.
- Ambiguous drug naming - generic vs. brand names, and local Sri Lankan brand variants, require normalisation (e.g. via RxNorm mappings) before retrieval.
- Responsible-use framing - dosage-related answers must consistently include safety disclaimers and avoid being presented as definitive clinical guidance.
- Evaluation data scarcity - constructing a representative QA evaluation set requires manual effort alongside the knowledge base curation.

Key References

- Lewis et al. (2020, NeurIPS) - Retrieval-Augmented Generation for knowledge-intensive NLP tasks (foundational RAG architecture).
- Singhal et al. (2023, Nature) - Large language models encode clinical knowledge; discusses risks of ungrounded medical LLM responses.
- OpenFDA API Documentation (U.S. FDA) - structured drug label data for cross-validation.
- WHO Model List of Essential Medicines (2023) - reference dosage and indication standards.

Evaluation Design

Layer 1 - Standard Metrics

Retrieval precision@k and recall@k for the vector search step; response latency; and exact/partial match accuracy against a held-out QA set derived from the knowledge base.

Layer 2 - Core Contribution

Groundedness audit: for a sample of responses, manually verify whether every factual claim in the answer is traceable to a retrieved knowledge-base chunk. Safety audit: confirm that 100% of dosage-related responses include the appropriate disclaimer and source citation.

Module 2 - AI-Augmented Reporting Agent

Domain: Automated Business Reporting / NLG | Overall Score: 7.3/10 | Strong Operational Value

Problem Statement

MediStore already generates scheduled and on-demand Daily/Weekly/Monthly Sales (PDF) and Inventory/Stock (CSV) reports. However, these reports present raw figures and tables without interpretation - pharmacy managers must manually identify trends, anomalies, and actionable items (e.g. unusually low sales of a category, a sudden spike in a particular medicine's demand, or items approaching reorder thresholds). This module proposes augmenting the existing reporting pipeline with an LLM-based narrative summarisation layer that converts structured report data into concise, actionable natural-language insights, without requiring any model training.

At a Glance

Task type	Structured-data-to-text summarisation and insight generation
Primary data source	MediStore's own operational data: sales transactions, inventory batches, stock levels, expiry dates (queried via existing service layer)
Models	No custom model training - LLM API (Claude/Gemini) prompted with structured JSON summaries of report data
Key technique	Prompt-templated narrative generation; trend detection via simple statistical comparison (current vs. previous period) before LLM summarisation
Output integration	AI-generated narrative section appended to existing PDF reports (ReportLab/WeasyPrint) and surfaced on the dashboard
Scheduling	Celery Beat - triggers AI summary generation alongside existing scheduled report jobs
Constraint	Numerical accuracy must be guaranteed by the underlying calculation (not the LLM); the LLM only narrates pre-computed figures

Objectives

- Extend the existing report-generation services to compute period-over-period comparisons (e.g. this week vs. last week sales by category, low-stock item counts, expiring-batch counts).
- Design a prompt template that converts this structured comparison data into a narrative summary highlighting notable trends, top/bottom performing items, and recommended actions.
- Implement the report_generator_agent as a FastAPI-internal module invoked by Celery tasks for scheduled reports and by an on-demand endpoint for ad-hoc summaries.
- Integrate the generated narrative into the existing PDF report templates as an 'AI Insights' section.
- Log all generated summaries (with their input data snapshot) for auditability and future evaluation.
- Evaluate summary quality and factual consistency against the source data through manual review by pharmacy-domain reviewers (e.g. project supervisor or pharmacy contacts).

Selection Factor Scores

Factor	Score	Rating
--------	-------	--------

Problem/gap relevance	7/10	Addresses a real but secondary pain point (interpretation, not data access)
Data availability	9/10	All data already exists within MediStore's own database
Technical complexity	5/10	Primarily prompt engineering and integration with existing report pipeline
Pharmacy operational impact	8/10	Saves manager time and surfaces trends that may otherwise be missed
Supervisor appeal	6/10	Practical, but technically less demanding than other modules
Novelty / contribution	5/10	Application of established LLM summarisation techniques to a new domain
Buildability in project time	9/10	Builds directly on existing reporting infrastructure
Literature depth	6/10	Data-to-text generation literature is mature; pharmacy-specific application is novel

Pre-Identified Challenges

- Numerical hallucination risk - the LLM must never recompute figures itself; all numbers must be injected into the prompt as pre-calculated values to avoid arithmetic errors.
- Defining 'notable' trends - thresholds for what constitutes a significant change (e.g. >15% deviation) need to be defined and tuned to avoid noisy or trivial insights.
- Cost and latency at scale - frequent scheduled summarisation across many report types may incur non-trivial LLM API costs; caching and batching strategies should be considered.
- Evaluation subjectivity - assessing the 'usefulness' of generated insights is inherently qualitative and requires structured reviewer feedback.

Key References

- Reiter & Dale (2000) - Building Natural Language Generation Systems (foundational data-to-text concepts).
- OpenAI / Anthropic prompt engineering documentation - structured-data summarisation patterns.
- Dou et al. (2021) - GSum: a general framework for guided neural abstractive summarization.

Evaluation Design

Layer 1 - Standard Metrics

Generation latency per report; cost per generated summary (API token usage); rate of successfully generated reports without errors across scheduled runs.

Layer 2 - Core Contribution

Factual consistency audit: for a sample of generated summaries, verify every numerical claim against the source data snapshot (target: 100% consistency, since figures are injected, not computed by the LLM).
Usefulness study: present AI-augmented vs. standard reports to a small group of reviewers and gather structured feedback on perceived decision-support value.

Module 3 - Report Analysis Agent (Document RAG)

Domain: Document Understanding / Retrieval-Augmented Generation | Overall Score: 7.5/10 | Already Prototyped

Problem Statement

Pharmacy staff and managers regularly receive external documents - vendor invoices, supplier price lists, regulatory circulars, and audit reports - that require manual review to extract relevant information. There is currently no mechanism within MediStore to query the contents of such documents directly. This module integrates a previously developed Retrieval-Augmented Generation (RAG) prototype - originally built as a standalone Python project ('AI Rag') with document loading, vector embedding (Qdrant), and query-answering components - into the unified MediStore FastAPI backend, allowing users to upload arbitrary documents and ask natural-language questions about their content directly from the React interface.

At a Glance

Task type	Document ingestion, chunking, embedding, and retrieval-augmented question answering
Primary input	User-uploaded documents (PDF, CSV, DOCX) - vendor invoices, supplier reports, regulatory documents
Existing components	data_loader.py (extraction & chunking), vector_db.py (Qdrant client wrapper), custom_types.py - already implemented and to be integrated
Vector store	Qdrant - per-user or per-document collections / metadata-filtered single collection
LLM	Claude API / Gemini API for answer synthesis from retrieved chunks
Key technique	Chunk-level semantic retrieval + context-grounded answer generation with source chunk references
Constraint	Multi-tenant data isolation - one user's uploaded reports must not be retrievable by another user's queries

Objectives

- Refactor the existing standalone RAG prototype (data_loader.py, vector_db.py, custom_types.py, main.py) into FastAPI-compatible modules under app/ai/rag/.
- Implement an upload endpoint (POST /api/v1/report-analysis/upload) that accepts a document, extracts and chunks its content, generates embeddings, and indexes them into Qdrant with user-scoped metadata.
- Implement a query endpoint (POST /api/v1/report-analysis/query) that embeds the user's question, retrieves the top-k relevant chunks for that user's documents, and generates a grounded answer.
- Add an uploaded_reports table to track upload metadata (filename, owner, collection/metadata reference, timestamp) for the React UI to list previously uploaded documents.
- Build a React interface (ReportAnalysis.jsx) for document upload and chat-style querying.
- Evaluate retrieval quality and answer accuracy on a test set of representative pharmacy documents with known question-answer pairs.

Selection Factor Scores

Factor	Score	Rating
Problem/gap relevance	8/10	Addresses a genuine document-handling bottleneck for pharmacy admin staff
Data availability	8/10	Test documents can be sourced from real or representative sample invoices/reports
Technical complexity	6/10	Core RAG logic already implemented; integration and multi-tenancy add complexity
Pharmacy operational impact	8/10	Reduces manual document review time significantly
Supervisor appeal	8/10	Demonstrates ability to integrate and productionise existing prototype work
Novelty / contribution	6/10	Novelty in domain application and integration architecture rather than core RAG technique
Buildability in project time	8/10	Substantial head start due to existing prototype
Literature depth	7/10	Strong RAG and document-QA literature base

Pre-Identified Challenges

- Multi-tenant isolation - ensuring Qdrant collection design (per-user collections vs. metadata filtering on a shared collection) correctly prevents cross-user data leakage.
- Heterogeneous document formats - invoices and reports may arrive as scanned PDFs, requiring OCR (pytesseract) as a fallback for the existing data_loader.py.
- Chunking strategy - financial/tabular documents (invoices, price lists) may require table-aware chunking rather than plain text splitting to preserve row-level context.
- Storage growth management - uploaded documents and their embeddings will accumulate over time; a retention/cleanup policy should be defined.
- Latency for large documents - embedding generation for large multi-page documents may introduce noticeable upload-to-ready delay; asynchronous processing (Celery) should be considered.

Key References

- Lewis et al. (2020, NeurIPS) - Retrieval-Augmented Generation for knowledge-intensive NLP tasks.
- Karpukhin et al. (2020, EMNLP) - Dense Passage Retrieval for open-domain question answering.
- Qdrant Documentation (2024/2025) - collection design and metadata filtering for multi-tenant vector search.

Evaluation Design

Layer 1 - Standard Metrics

Retrieval precision@k / recall@k on a labelled test set of (document, question, relevant chunk) triples; end-to-end answer accuracy (exact/partial match) against expected answers; ingestion success rate

across document formats (PDF, CSV, DOCX, scanned PDF with OCR).

Layer 2 - Core Contribution

Multi-tenancy isolation test: verify across a sample of cross-user queries that no chunks belonging to another user's documents are ever retrieved. Integration audit: compare end-to-end latency and accuracy of the integrated FastAPI implementation against the original standalone prototype to confirm no regression from refactoring.

Module 4 - Pharmacy Business Intelligence & Assistant Agent

Domain: Tool-Augmented LLM Agents / Decision Support | Overall Score: 8.0/10 | Highest Recommended Priority

Problem Statement

MediStore already collects rich operational data - inventory levels, batch expiry dates, sales history, vendor restock records - but extracting actionable insight requires staff to manually navigate multiple dashboard views and cross-reference figures (e.g. 'which items are both low in stock and have had increasing demand over the past month?'). This module proposes a conversational Business Intelligence agent that uses LLM function-calling (tool use) to query MediStore's existing service layer directly, reason over the combined results, and provide natural-language answers and recommendations - effectively acting as an on-demand pharmacy operations analyst.

At a Glance

Task type	Tool-augmented conversational agent (LLM function-calling) over live operational data
Data source	MediStore's own database via existing service-layer functions - no external dataset required
Tools exposed to agent	get_low_stock_items(), get_expiring_items(days), get_sales_summary(start,end), get_reorder_suggestions(), get_top_selling_items(period)
LLM	Claude API / Gemini API with function-calling / tool-use support
Key technique	Multi-turn tool-calling agent loop: LLM selects tool(s) -> internal async function executes against the database -> results returned to LLM -> LLM synthesises final natural-language response
Conversation memory	Redis-backed conversation history keyed by conversation_id for multi-turn context
Constraint	Tool functions must enforce the same RBAC permissions as the underlying REST endpoints (e.g. Staff role should not access financial summaries reserved for Admin)

Objectives

- Define and implement a set of internal 'tool' functions (medistore_tools.py) wrapping existing inventory, sales, and reorder service functions for safe read-only access.
- Implement the pharmacy_bi_agent using the LLM provider's function-calling API, including tool definitions, the agent loop (tool selection -> execution -> response synthesis), and error handling for failed tool calls.
- Integrate role-based access control into the tool layer so that the agent only exposes tools/data appropriate to the requesting user's role.
- Implement conversation context persistence (Redis) to support multi-turn follow-up questions.
- Build a chat interface (AIAssistant.jsx) in React for natural-language interaction with the agent.

- Evaluate the agent's tool-selection accuracy, response correctness, and usefulness across a set of representative pharmacy-management queries.

Selection Factor Scores

Factor	Score	Rating
Problem/gap relevance	8/10	Directly automates cross-referencing tasks staff currently do manually
Data availability	9/10	Entirely powered by MediStore's own existing database - zero external data needed
Technical complexity	6/10	Function-calling agent loops require careful design but are well-documented
Pharmacy operational impact	9/10	Highest direct operational value of all proposed modules
Supervisor appeal	8/10	Demonstrates modern agentic AI design patterns (tool use, RBAC-aware agents)
Novelty / contribution	7/10	Novel application of tool-calling agents to pharmacy operations management
Buildability in project time	8/10	No data collection required; builds on existing service layer
Literature depth	7/10	Growing literature on LLM agents and tool use, though pharmacy-specific applications are scarce - a contribution opportunity

Pre-Identified Challenges

- RBAC enforcement at the tool layer - the agent must not be able to bypass role restrictions by calling tools directly; each tool function must independently verify the requesting user's permissions.
- Tool-selection reliability - the LLM may occasionally select an inappropriate tool or fail to combine multiple tool results correctly; prompt design and tool descriptions must be carefully tuned.
- Hallucinated recommendations - reorder suggestions and BI insights generated by the agent must be clearly framed as suggestions requiring human approval, not automated actions.
- Conversation state management - multi-turn context via Redis introduces session-expiry and memory-size considerations.
- Cost management - complex multi-tool queries may require multiple LLM round-trips, increasing latency and API cost per query.

Key References

- Schick et al. (2023) - Toolformer: Language models can teach themselves to use tools.
- Yao et al. (2023, ICLR) - ReAct: Synergizing reasoning and acting in language models.
- Anthropic / OpenAI function-calling / tool-use API documentation (2024/2025).
- Wang et al. (2024) - A survey on large language model based autonomous agents.

Evaluation Design

Layer 1 - Standard Metrics

Tool-selection accuracy (does the agent call the correct tool(s) for a given query, evaluated against a labelled set of test queries); end-to-end response correctness against ground-truth database values; average response latency and number of tool calls per query.

Layer 2 - Core Contribution

RBAC compliance audit: for each user role (Admin, Pharmacist, Staff), verify across a test query set that the agent never returns data the role is not permitted to access. Decision-support utility study: present a set of complex multi-factor queries (e.g. combined low-stock + expiry + sales-trend questions) to reviewers and compare time-to-insight using the agent versus manual dashboard navigation.

Module 5 - Face Recognition-Based Staff Attendance System

Domain: Computer Vision / Biometric Attendance | Overall Score: 6.8/10 | Prototype Already Developed

Problem Statement

Manual attendance tracking (sign-in sheets or manual admin entry) is time-consuming and prone to proxy attendance (one staff member marking another as present). A biometric, face-recognition-based attendance system removes this risk and reduces administrative overhead. A working prototype - face detection via webcam combined with a face recognition classifier trained using Teachable Machine - has already been developed. This module proposes integrating that prototype into the unified MediStore FastAPI backend as a formal Attendance module, including database persistence, RBAC-aware reporting, and a React-based dashboard.

At a Glance

Task type	Real-time face detection + face recognition (closed-set classification) for attendance marking
Existing components	Webcam-based face detection prototype; Teachable Machine-trained recognition model (exported as Keras/.h5 or TFLite)
Detection	OpenCV (Haar Cascade or DNN-based face detector) for locating faces in webcam frames
Recognition	TensorFlow Lite inference using the existing Teachable Machine-exported model
Backend integration	FastAPI endpoint POST /api/v1/attendance/recognize - accepts a frame, runs detection + recognition, and records attendance
Database additions	attendance table (user_id, check_in_time, check_out_time, confidence_score, status); staff_face_profile mapping (model label -> user_id)
Constraint	Confidence thresholding to avoid false-positive attendance marking; privacy considerations around biometric data storage

Objectives

- Migrate the existing face detection and Teachable Machine recognition code into a face_attendance module within the unified FastAPI backend (face_detector.py, face_recognizer.py).
- Design and implement the attendance and staff_face_profile database tables, including a mapping between Teachable Machine class labels and MediStore user accounts.
- Implement the /attendance/recognize endpoint: accepts a captured frame (base64), performs face detection, crops the face region, runs recognition inference, and applies a confidence threshold before marking check-in/check-out.
- Implement attendance_processor logic to determine check-in vs. check-out based on the staff member's existing attendance record for the current day.
- Build a React component (WebcamCapture.jsx) for periodic frame capture at a kiosk/entrance device, and an AttendanceDashboard.jsx for Admin to view and export attendance records.
- Evaluate recognition accuracy, false-acceptance/false-rejection rates, and real-world reliability under varying lighting conditions.

Selection Factor Scores

Factor	Score	Rating
Problem/gap relevance	7/10	Addresses a genuine administrative pain point (manual attendance, proxy marking)
Data availability	7/10	Training data is staff face images (already collected for the prototype); limited by small staff cohort size
Technical complexity	6/10	Detection + recognition pipeline integration, plus real-time/edge considerations
Pharmacy operational impact	7/10	Improves attendance accuracy and reduces admin workload
Supervisor appeal	6/10	Demonstrates CV integration; technique itself (Teachable Machine) is relatively simple
Novelty / contribution	5/10	Limited novelty in the recognition technique; contribution is the system integration and attendance-logic design
Buildability in project time	8/10	Prototype already exists - integration-focused remaining work
Literature depth	5/10	Face recognition literature is extensive but Teachable Machine-based approaches are rarely covered in academic literature

Pre-Identified Challenges

- Confidence threshold tuning - setting an appropriate threshold to balance false rejections (legitimate staff not recognised) against false acceptances (incorrect identification).
- Lighting and camera angle variability - the Teachable Machine model's robustness under varying real-world conditions (vs. controlled training conditions) needs validation and may require retraining with more diverse samples.
- Small dataset / closed-set limitation - Teachable Machine models are trained on a small, fixed set of staff members; adding/removing staff requires retraining the model, which should be documented as an operational limitation.
- Privacy and data protection - storing facial recognition data and attendance logs requires appropriate consent processes and secure storage, to be documented in the project's ethical considerations.
- Duplicate/rapid-fire marking - logic must prevent multiple check-ins being recorded from consecutive frames within a short time window.

Key References

- Viola & Jones (2001) - Rapid object detection using a boosted cascade of simple features (basis of Haar Cascade face detection).
- Google Teachable Machine Documentation (2024) - model export formats and deployment guidance.
- Wang et al. (2021) - Deep face recognition: a survey.

Evaluation Design

Layer 1 - Standard Metrics

Recognition accuracy, false-acceptance rate (FAR), and false-rejection rate (FRR) measured on a test set of staff face images captured under varied lighting/angle conditions; end-to-end latency from frame capture to attendance record creation.

Layer 2 - Core Contribution

Field trial: deploy the system for a trial period (e.g. one to two weeks) with the project team acting as test 'staff', comparing system-recorded attendance against a manually-kept ground-truth log to assess real-world reliability and identify edge cases (e.g. staff wearing masks/glasses).

Consolidated Technology Stack

The table below summarises the complete technology stack proposed for MediStore AI, consolidating the existing platform components with the additions required across Modules 1-5.

Layer	Technology	Purpose
Frontend	React	Existing UI; extended with AI chat, report-analysis, and attendance dashboards
Backend Framework	Python 3.12 + FastAPI	Unified backend for all modules (replacing Spring Boot)
Relational Database	PostgreSQL / MySQL + SQLAlchemy (async) + Alembic	Core transactional data: users, inventory, sales, attendance, AI logs
Vector Database	Qdrant	Embeddings for medicine knowledge base (Module 1) and uploaded reports (Module 3)
LLM Provider	Claude API / Gemini API / Groq	Powers Modules 1, 2, 3 and 4 (Q&A, summarisation, RAG, tool-calling)
Background Jobs	Celery + Redis	Scheduled report generation, email alerts, async document processing
Computer Vision	OpenCV + TensorFlow Lite	Face detection and Teachable Machine-based recognition (Module 5)
PDF/CSV Generation	ReportLab / WeasyPrint + pandas	Existing and AI-augmented report output
Auth & Security	JWT (python-jose) + Passlib (bcrypt)	Authentication and RBAC, enforced at both API and AI tool layers
Containerisation	Docker Compose	API, Celery worker/beat, Postgres, Redis, Qdrant, frontend services

Summary & Supervisor Guidance Request

All five proposed modules build directly on the existing MediStore platform and require no new external data collection beyond the curated medicine knowledge base for Module 1 and the existing face recognition prototype for Module 5. The author is confident in the ability to implement all five modules within the standard project timeline, with Modules 3 (Report Analysis Agent) and 5 (Face Recognition Attendance) benefiting from substantial existing prototype work, and Module 4 (Pharmacy BI Agent) identified as the highest-impact contribution.

The following questions are respectfully submitted for the supervisor's feedback:

- Does the proposed scope (five integrated AI modules plus a backend re-architecture from Spring Boot to FastAPI) represent an appropriate breadth for a single FYP, or should the project focus more narrowly on a subset of modules (e.g. Modules 1, 3, and 4) with the remainder positioned as future work?
- Is the proposed approach of using third-party LLM APIs (rather than training custom models) considered acceptable and sufficiently rigorous for FYP evaluation, given that the system integration, RAG architecture, RBAC-aware tool design, and evaluation methodology constitute the core technical contribution?
- For Module 1 (Medicine Q&A Agent), would the department be able to recommend a verified Sri Lankan or regional drug formulary reference to strengthen the knowledge base beyond international sources (OpenFDA, WHO EML)?
- For Module 5 (Face Recognition Attendance), are there any data protection / ethics clearance procedures the project should follow given the use of staff biometric data, even within a university-internal trial?
- Is the proposed two-layer evaluation design (standard technical metrics plus a contribution-specific evaluation, e.g. groundedness audits, RBAC compliance audits, and field trials) considered appropriate, or are there additional evaluation dimensions the supervisor would recommend?

Thank you for your time and guidance. The author looks forward to your feedback and is available to meet at your convenience to discuss any of the proposed modules in detail.